SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:25**

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

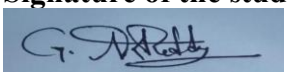
Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Code of Conduct:

I **G Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Question 1: CNN vs Fully Connected Neural Network

Q1: A medical imaging organization wants to classify X-ray images using deep learning. Compare a CNN with a traditional fully connected neural network in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

1. Selection of Parameters

The comparison uses six parameters directly relevant to X-ray image classification: architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. These parameters were chosen because they collectively determine whether a network can learn medically meaningful patterns (e.g., fractures, opacities) efficiently and without overfitting on limited medical datasets.

2. Comparison & Differentiation

Parameter	Fully Connected Neural Network (FCNN)	Convolutional Neural Network (CNN)
Architectural Design	Every neuron in one layer connects to every neuron in the next; the 2D image must be flattened into a 1D vector, destroying spatial layout.	Neurons connect only to a local receptive field; the image is processed as a 2D/3D grid, preserving spatial structure.
Motivation	Designed for generic tabular/vector data where feature position carries no inherent meaning.	Designed specifically for grid-like data (images) where nearby pixels are strongly correlated and local patterns matter.
Spatial Feature Extraction	None — flattening removes adjacency information, so the network cannot exploit that a tumor's pixels are spatially clustered.	Convolutional filters slide over the image and explicitly extract edges, textures, and shapes while preserving location.
Number of Parameters	Extremely high — a 256x256 X-ray flattened to 65,536 inputs connected to even 500 hidden neurons needs ~32.8 million weights in one layer alone.	Low — a 3x3 filter with 32 channels needs only ~288 weights per layer, reused (shared) across the entire image.
Computational Complexity	High memory and compute cost due to dense connections; scales poorly with image resolution.	Lower compute cost per layer due to weight sharing and sparse local connections; scales better with resolution using pooling/stride.
Ability to Handle Image Data	Poor — loses translation invariance, prone to overfitting, cannot generalize to shifted/rotated anatomical structures.	Strong — learns hierarchical, translation-invariant features suited to detecting abnormalities anywhere in the X-ray.

3. Role of Convolutional Layers, Filters, and Pooling Layers

- Convolutional layers: apply learnable filters across the image to produce feature maps highlighting edges, bone boundaries, and tissue density variations without discarding spatial context.
- Filters (kernels): small weight matrices (e.g., 3x3, 5x5) that are shared across the entire image; early filters detect low-level features (edges, gradients) while deeper filters combine these into high-level structures (rib patterns, lesion shapes).
- Pooling layers (max/average pooling): downsample feature maps, reducing spatial dimensions and parameter count while retaining the strongest activations, which adds a degree of translation invariance and reduces overfitting risk on limited medical data.

4. Critical Evaluation

FCNNs are simple to implement but structurally unsuited to imaging tasks: the explosion in parameter count with an FCNN increases the risk of overfitting on typically small, expensively-labeled medical datasets, and the loss of spatial information means an FCNN cannot inherently recognize the same abnormality if it appears in a different part of the X-ray. CNNs address both weaknesses through parameter sharing and local connectivity, but they are not without cost — deeper CNNs need larger annotated datasets and GPU acceleration to train effectively, and interpretability of learned filters can be limited, which is a genuine concern in clinical decision-making. This trade-off is why explainability techniques (e.g., Grad-CAM) are often paired with CNNs in medical imaging.

5. Synthesis & Insight

The core insight is that CNNs encode a useful inductive bias — locality and translation invariance — that matches the statistical structure of images, whereas FCNNs treat every pixel as an independent, position-specific feature. For X-ray classification, this bias directly translates into fewer parameters, lower overfitting risk, and better generalization to abnormalities appearing at varying locations, which is why CNNs (not FCNNs) are the practical choice for medical imaging pipelines.

6.Conclusion / Justification

CNNs are more suitable than fully connected networks for X-ray classification because they exploit spatial locality through convolution and parameter sharing, drastically reduce parameter count and computational load, and produce translation-invariant, hierarchical features — all essential for reliably detecting medical abnormalities regardless of their position in the image.

Question 2: Different CNN Layers and Filters

Q2: A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of convolutional layers, pooling layers, activation layers, and fully connected layers in terms of feature extraction, spatial information, computational requirements, and classification. Further compare early-layer filters and deeper-layer filters based on the type of features they learn, and justify how these layers collectively improve defect detection.

1. Selection of Parameters

Four core CNN layer types are compared — convolutional, pooling, activation, and fully connected — using four evaluation dimensions (feature extraction, spatial information retention, computational requirements, and role in classification) since these determine how raw pixel data from product images is transformed into a defect/no-defect decision.

2. Comparison & Differentiation — Layer Functions

Layer Type	Feature Extraction	Spatial Information	Computational Requirements	Role in Classification
Convolutional	Extracts local patterns (scratches, cracks, dents) via learnable filters.	Fully preserved — output feature maps retain 2D spatial layout.	Moderate; reduced further by weight sharing versus dense layers.	Provides the discriminative feature maps used downstream.
Pooling	No new features learned; aggregates/summarizes existing ones (max or average).	Reduced resolution but coarse spatial layout retained.	Very low — no learnable parameters, only a downsampling operation.	Improves robustness to minor shifts/noise in defect location.
Activation (e.g., ReLU)	Introduces non-linearity so the network can model complex, non-linear defect patterns.	Preserved — applied element-wise, does not alter spatial layout.	Negligible computational cost.	Enables the network to separate non-linearly separable defect classes.
Fully Connected	Combines high-level features from all spatial locations into global representations.	Discarded — flattens feature maps into a 1D vector.	High — dense connections dominate parameter count near the output.	Performs the final defect classification decision.

3. Early-Layer Filters vs Deeper-Layer Filters

Aspect	Early-Layer Filters	Deeper-Layer Filters
Type of Features Learned	Low-level, generic features: edges, corners, color blobs, simple textures.	High-level, task-specific features: complex shapes, defect patterns, object parts.
Receptive Field	Small — sees a limited local region of the image.	Large — aggregates information across a wide area via stacked convolutions/pooling.
Generality	Highly generic; similar across most CNNs and datasets (useful for transfer learning).	Highly specific to the trained task, e.g., a particular scratch or crack morphology.
Relevance to Defect Detection	Detects raw edges of a scratch or the boundary of a dent.	Recognizes the complete defect pattern and distinguishes defect types.

4. Critical Evaluation

Each layer type addresses a different weakness of the others: convolution alone would produce huge, redundant feature maps without pooling; pooling alone destroys detail without preceding convolutions to first extract meaningful patterns; and without non-linear activation the entire stack would collapse mathematically into a single linear transformation, unable to model the complex boundaries between defective and non-defective surfaces. The fully connected layer is powerful for final classification but is also the most parameter-heavy and overfitting-prone component, so in practice it is kept shallow (often 1–2 layers) with dropout applied, relying on the convolutional backbone to do the heavy representational work.

5. Synthesis & Insight

The layers form a deliberate pipeline: convolution + activation extract and non-linearly transform local patterns, pooling compresses and stabilizes them, and this repeats to build a feature hierarchy from edges to full defect signatures, which the fully connected layer finally maps to a classification decision. This hierarchical design is precisely what allows small, subtle manufacturing defects to be captured by early filters and correctly categorized by deeper filters.

6.Conclusion / Justification

Convolutional, pooling, activation, and fully connected layers each contribute a distinct, complementary function; together they let the network progress from raw pixel-level edges (early layers) to abstract, defect-specific patterns (deeper layers), enabling accurate and computationally efficient automated defect detection on the production line.

Question 3: Parameter Sharing vs Independent Weights

Q3: A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare parameter sharing in CNNs with independent weights in fully connected networks in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

1. Selection of Parameters

The comparison focuses on number of parameters, memory requirements, computational complexity, feature detection capability, and translation invariance — the five factors that most directly affect whether a network can scale to processing thousands of high-resolution surveillance frames in real time.

2. Comparison & Differentiation

Parameter	Independent Weights (Fully Connected)	Parameter Sharing (CNN)
Number of Parameters	Extremely high — a separate weight exists for every input-output neuron pair; grows quadratically with image size.	Very low — the same filter (e.g., 3x3x3) is reused at every spatial location, independent of image size.
Memory Requirements	Very high; storing weight matrices for high-resolution frames (e.g., 1920x1080) is often infeasible.	Low; only filter weights are stored, not per-pixel connections, making high-resolution processing practical.
Computational Complexity	High — dense matrix multiplications scale with the full input size at every layer.	Lower — convolution operations scale with filter size and stride, not raw resolution, especially with pooling.
Feature Detection	Learns a unique, position-specific response for every location; cannot generalize a pattern learned in one location to another.	Learns a single generalized detector (e.g., 'edge' or 'motion boundary') applied uniformly across the whole frame.
Translation-Related Invariance	None — a person detected in the top-left corner produces different weights/response than the same person in the bottom-right.	Strong — the same filter detects a person or object regardless of where it appears in the frame.

3. How Weight Reuse Improves Learning Efficiency

- Fewer parameters to learn means faster convergence and lower risk of overfitting, even when training data (labeled surveillance footage) is limited relative to the number of possible object positions.
- A pattern learned from one region of an image (e.g., a person's silhouette in one frame position) automatically generalizes to all other positions, effectively multiplying the training signal without needing more labeled examples.
- Reduced memory footprint allows deeper networks and higher-resolution inputs to fit within GPU memory, which is essential when processing multiple concurrent high-resolution camera feeds.

4. Critical Evaluation

Parameter sharing is not a free advantage: it assumes that the same feature is equally relevant everywhere in the image, which holds well for surveillance object detection but can be a limitation when spatial position itself carries information (e.g., a fixed camera where only a specific zone is relevant), sometimes requiring positional encodings or region-of-interest cropping as a supplement. Independent weights, in contrast, could in theory model position-specific patterns exactly, but this benefit is far outweighed by the impracticality of their memory and compute costs at surveillance-scale resolutions, and by their tendency to overfit.

5. Synthesis & Insight

The essential insight is that parameter sharing converts a computationally intractable problem (independent weights for millions of pixel positions) into a tractable one by exploiting the fact that useful visual features are largely position-independent. This is precisely the property that allows a single trained CNN to be deployed efficiently across thousands of high-resolution frames without a proportional explosion in model size or inference cost.

6.Conclusion / Justification

For large-scale, high-resolution image processing such as smart surveillance, parameter sharing in CNNs is essential: it keeps parameter count and memory usage low, reduces computational complexity, and provides translation invariance, allowing the same trained filters to reliably detect objects or events anywhere in the frame across thousands of images.

Question 4: AlexNet vs ResNet

Q4: A computer vision company is developing an image classification system and is considering AlexNet and ResNet as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

1. Selection of Parameters

Nine parameters are used — network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient behavior, computational cost, and classification performance — covering both architectural structure and practical trainability, which together determine suitability for complex, large-scale image recognition.

2. Comparison & Differentiation

Parameter	AlexNet	ResNet
Network Depth	Shallow by modern standards — 8 weight layers (5 convolutional + 3 fully connected).	Very deep — common variants have 18, 34, 50, 101, or even 152+ layers.
Layer Organization	Sequential stack of conv, ReLU, and max-pooling layers followed by large fully connected layers.	Organized into residual blocks with skip (shortcut) connections that add each block's input to its output.
Convolutional Filters	Uses large filters in early layers (11x11, 5x5) with large strides.	Uses small, uniform filters (mostly 3x3) stacked in depth, following a more efficient design.
Parameter Requirements	~60 million parameters, dominated by the large fully connected layers.	Fewer parameters despite far greater depth (e.g., ResNet-50 has ~25 million) due to smaller filters and no huge FC layers.
Feature Extraction Capability	Good general-purpose features but limited by shallow depth.	Much richer hierarchical feature extraction enabled by depth without degradation.
Training Difficulty	Relatively easier to train given shallower depth, but prone to overfitting without heavy regularization/data augmentation.	Deeper networks are traditionally harder to train, but residual connections make ResNet's optimization notably easier than a plain deep network of similar depth.
Vanishing-Gradient Problem	Present and worsens with depth, limiting how deep AlexNet-style plain networks can practically go.	Substantially mitigated by skip connections, which allow gradients to flow directly through identity shortcuts during backpropagation.
Computational Cost	Lower raw depth but heavy in FC-layer computation and memory.	Higher total layer count but efficient per-layer cost due to small filters and bottleneck designs.
Classification Performance	Strong for its era (ILSVRC-2012 winner) but surpassed by later, deeper architectures.	State-of-the-art-level performance on large benchmarks, generally outperforming AlexNet in accuracy.

3. Critical Evaluation

AlexNet's historical significance lies in demonstrating that deep CNNs trained on GPUs could dramatically outperform traditional methods, but its architecture — particularly the reliance on massive fully connected layers — makes it

parameter-inefficient and limits how much depth (and therefore representational power) it can practically add before gradients vanish. ResNet's residual connections directly solve this depth-vs-trainability trade-off: by learning residual mappings instead of direct mappings, each block only needs to learn a small correction to its input, keeping gradients well-behaved even across very deep stacks. The trade-off is that ResNet's greater depth generally requires more total computation and longer training time, and its benefits are most pronounced on large, complex datasets — on very small or simple datasets the extra depth may be unnecessary.

4. Synthesis & Insight

The key architectural insight is that depth is valuable for feature richness but only if the network remains trainable; AlexNet increased depth relative to prior methods but hit practical limits, while ResNet's shortcut connections decoupled depth from the vanishing-gradient problem, unlocking substantially deeper and more accurate networks without a proportional rise in parameters.

5. Recommendation & Justification

For a complex image recognition application, ResNet is the more suitable architecture. Its residual connections allow much greater depth without succumbing to vanishing gradients, its small, uniform filters keep parameter count efficient relative to its depth, and its hierarchical feature extraction consistently yields higher classification performance on complex, large-scale datasets than AlexNet's shallower, FC-heavy design.

Question 5: Regularized CNN vs Non-Regularized CNN

Q5: An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a CNN without regularization and a CNN using dropout, data augmentation, and batch normalization in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

1. Selection of Parameters

The comparison uses overfitting behavior, generalization to unseen data, training stability, computational requirements, and overall model performance — directly diagnosing the reported symptom (high training accuracy, poor unseen-image accuracy) which is a textbook indicator of overfitting.

2. Comparison & Differentiation

Parameter	Non-Regularized CNN	Regularized CNN (Dropout + Augmentation + Batch Norm)
Overfitting	High — the model memorizes training-specific leaf textures, backgrounds, and lighting rather than general disease patterns.	Reduced — each technique disrupts memorization pathways or expands effective training diversity.
Generalization	Poor on unseen images, exactly matching the described symptom of high train / low test accuracy.	Improved — the model learns disease features that transfer across leaf orientation, lighting, and background variation.
Training Stability	Can be unstable across layers/batches, with internal covariate shift making optimization sensitive to learning rate and initialization.	More stable — batch normalization normalizes layer inputs, allowing higher learning rates and smoother convergence.
Computational Requirements	Lower per-epoch cost, but often needs many more epochs or trial-and-error tuning to fight overfitting indirectly.	Slightly higher per-epoch cost (extra operations, augmented data) but more efficient overall due to faster, more reliable convergence.
Model Performance (Real-World)	Misleadingly good on training data but unreliable in the field on new leaf images.	Consistently better real-world performance, better suited to deployment on farms with varied imaging conditions.

3. Which Regularization Techniques Should Be Applied, and Why

- Data Augmentation: rotate, flip, crop, and adjust brightness/contrast of leaf images to simulate real-world variation (different angles, lighting, camera quality), directly increasing effective dataset diversity and reducing memorization of specific training images.
- Dropout: randomly deactivate a fraction of neurons during training so the network cannot rely on any single feature detector, forcing it to learn redundant, robust representations of disease symptoms (e.g., spot patterns, discoloration) rather than incidental artifacts.
- Batch Normalization: normalize activations within each mini-batch, stabilizing and accelerating training, and providing a mild regularizing effect by adding noise proportional to batch statistics.

4. Critical Evaluation

No single technique fully resolves the overfitting problem in isolation: data augmentation increases input diversity but does not directly constrain the network's internal representations; dropout regularizes the network internally but can slow convergence if applied too aggressively; and batch normalization primarily improves training stability, with its

regularization effect being a secondary benefit. Using all three together is more effective because they address overfitting from complementary angles — input diversity, internal redundancy, and stable optimization — though care must be taken with hyperparameters (dropout rate, augmentation intensity) since excessive regularization can underfit and reduce training accuracy too much.

5. Synthesis & Insight

The described symptom — high training accuracy but poor performance on unseen leaf images — is a classic overfitting signature, meaning the model has learned dataset-specific noise rather than the actual disease-relevant patterns. Combining data augmentation, dropout, and batch normalization systematically closes this gap by simultaneously diversifying the training signal, discouraging co-adapted/memorized features, and stabilizing the learning process.

6.Conclusion / Justification

For reliable real-world plant disease classification, the regularized CNN — using data augmentation, dropout, and batch normalization together — is the most appropriate approach. It directly targets the reported overfitting problem, improves generalization to new, unseen field images, and provides more stable training, making it far better suited to deployment than the non-regularized model.